

Linguaggi di Programmazione e Modelli Computazionali

Esame del 20 Febbraio 2014

Esercizio 1:

Si mostri un'espressione regolare che genera il linguaggio sull'alfabeto $\{0,1\}$ in cui il numero di occorrenze di 0 ed il numero di occorrenze di 1 siano entrambi pari oppure entrambi dispari.

X Esercizio 2:

Si mostri una grammatica libera dal contesto che genera il linguaggio delle stringhe palindrome sull'alfabeto binario $\{0,1\}$ in cui il numero di occorrenze di 0 risulta essere un multiplo di 4.

X Esercizio 3:

Si consideri il linguaggio delle stringhe palindrome sull'alfabeto binario $\{0,1\}$ in cui il numero di occorrenze di 1 è strettamente maggiore del numero di occorrenze di 0 . Classificare il linguaggio dicendo se è un linguaggio regolare, libero, ricorsivo, ~~ricorsivamente~~ enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

Esercizio 4:

Si considerino due generici linguaggi ricorsivamente enumerabili $L1$ e $L2$ ed il seguente corrispondente linguaggio:

$$L = \{w \mid w \text{ appartiene a } L1 \text{ ed è la concatenazione di due stringhe di } L2\}$$

Dire se L è ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

X Esercizio 5:

Se esiste, descrivere un algoritmo capace di verificare l'appartenenza di una stringa ad un linguaggio libero dal contesto. In caso contrario, giustificare perché tale algoritmo non esiste.

X Esercizio 6:

Si consideri la grammatica libera dal contesto:

$$S \rightarrow CC \quad C \rightarrow cC \mid d$$

Dire se la grammatica è di tipo LR(1) e nel caso mostrare la tabella di parsing.

X Esercizio 7:

Si consideri un sistema del tipo "dining philosopher" con 3 filosofi e darne una rappresentazione formale tramite rete di Petri. Descrivere tramite LTL la principale proprietà di tale sistema, cioè che due filosofi non potranno mai mangiare contemporaneamente.

Esercizio 1) $L \rightarrow \text{Regex}$

$$L = \{ w \in \{0,1\}^* \mid (\#1_{\text{peri}} \text{ AND } \#0_{\text{peri}}) \text{ OR } (\#1_{\text{dupen}} \text{ AND } \#0_{\text{dupen}}) \}$$

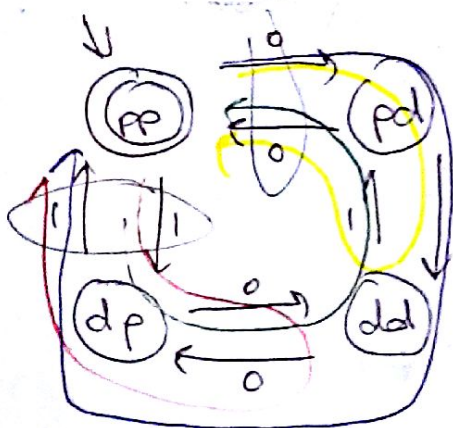
$$L = L_1 \cup L_2 \quad \text{NB. l'unione di regole è regolare}$$

$$L_1 = \{ w \in \{0,1\}^* \mid \#1_{\text{peri}} \text{ AND } \#0_{\text{peri}} \}$$

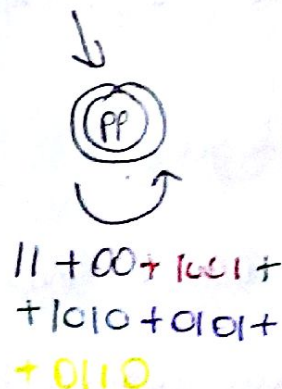
$$L_2 = \{ w \in \{0,1\}^* \mid \#1_{\text{dupen}} \text{ AND } \#0_{\text{dupen}} \}$$

$$\text{NB. } L_1 \cap L_2 = \emptyset$$

$$L_1: \Rightarrow R_1 = R_1 + R_2 \quad \text{Calcolo } R_1 \text{ e } R_2:$$

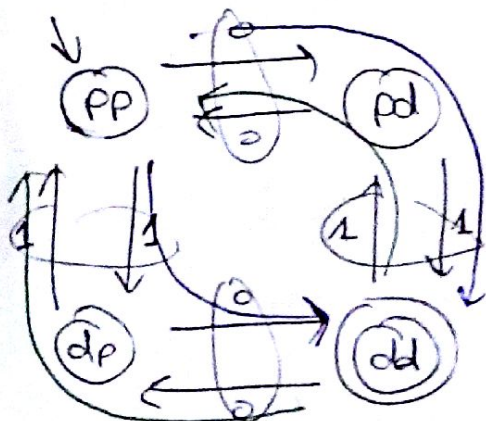


$\Rightarrow$

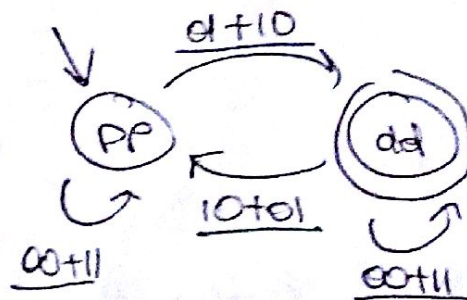


$$\Rightarrow R_1 = (11 + 00 + 1001 + 1010 + 0101 + 0110)^*$$

$$L_2: \underline{\hspace{10cm}}$$



$\Rightarrow$



$$R_2 = ((00+11) + (01+10)(00+11)^*(10+01))^*(10+01)(00+11)^*$$

Es 2) $L \rightarrow$ CFG

$$L = \{w \in \{0,1\}^* \mid w \text{ è palindromo AND } \#0 \bmod 4 = 0\}$$

CFG per stringhe palindrome: $S \rightarrow 1S1 \mid 0S0 \mid 1 \mid 0 \mid \epsilon$

NB. le stringhe palindromo possono avere lunghezza dispari?

Adesso devo modificarlo per far sì che gli zeri siano sempre multipli di 4

$$\Rightarrow S \rightarrow 1S1 \mid 00S00 \mid 1 \mid \cancel{0000} \mid \epsilon \mid \cancel{0101} \mid \cancel{1010} \mid 010S010$$

poi mi accorgo che 0000 può essere generata dalle seconde e quarta produzioni

$$\Rightarrow S \rightarrow 1S1 \mid 00S00 \mid 1 \mid \epsilon \mid \cancel{0101} \mid \cancel{1010} \mid 010S010$$

Es. 3) Classificazione linguaggio $\uparrow$ si genera da $010S010 \rightarrow 1S1$

$$L = \{w \in \{0,1\}^* \mid w \text{ è palindromo AND } \#1 > \#0\}$$

Intuizione: E' libero poiché posso costruire una grammatica CFG G :
 $L(G) = L$.

Passo 1: Dimostro che non è regolare $\Rightarrow$ PL regolari:

$$\text{Def } I_n: \forall w \in L, |w| \geq n \quad w = z_1 z_2 z_3$$

$$\textcircled{I} |z_1 z_2| \leq n$$

$$\textcircled{II} |z_2| \geq 1 \Rightarrow z_2 \neq \epsilon$$

$$\textcircled{III} z_i = z_1 (z_2)^i z_3 \in L \quad \forall i \in \mathbb{N}$$

Considero $w = \underbrace{1^n}_\text{primo gruppo} 0 \underbrace{1^{2n}}_\text{secondo gruppo}$ $|w| = 4n + 1 > n$

per la (I) devo scegliere z_2 composto di soli 1 del primo gruppo
 ma così facendo $\forall i \in \mathbb{N} - \{1\} z_i \notin L$ perché non è palindroma
 $\Rightarrow$ contraddizione.

PASSO 2: Dimostro che L è libero progettando una grammatica context-free che genera L .

$S \rightarrow 1S1 \mid 01S10 \mid 10S01 \mid 1 \mid 101 \mid 11$

Problema! Non riesco a generare $0^n 1^n 11^n 0^n$

Sospetto che forse non sia neanche libero! Provo col PL dei Liberi

$\exists n: \forall z \in L: |z| \geq n \quad z = z_1 z_2 z_3 z_4 z_5$

$$\textcircled{I} |z_2 z_3 z_4| \leq n$$

$$\textcircled{II} |z_2 z_4| \geq 1$$

$$\textcircled{III} z_i = z_1 (z_2)^i z_3 (z_4)^i z_5 \in L \quad \forall i \in \mathbb{N}$$

Considero $z = \underbrace{0^n}_{\text{gruppo 1}} \underbrace{1^n}_{\text{gruppo 2}} \underbrace{1}_{\text{uno centrale}} \underbrace{1^n}_{\text{gruppo 3}} \underbrace{0^n}_{\text{gruppo 4}}$ $|z| = 4n + 1 > n$

La condizione (III) ed il vincolo di palindromia mi costringono a scegliere z_2 nel gruppo 2 e z_4 nel gruppo 3, altrimenti si viola la palindromia. In ogni altro caso la P.L. è violata.
~~Le condizioni~~ Sono anche costretto a scegliere $|z_2| = |z_4| \geq 0$ in ogni altro caso la palindromia è violata.

L'unico caso utile è quando $z_2 = \text{ultimo 1 del gruppo 2}$
 $z_3 = \text{uno centrale}$
 $z_4 = \text{primo 1 del gruppo 3}$

In questo caso per $i=0$ la P.L. è violata.

$\Rightarrow$ non esiste una scelta di $z_1, \dots, z_5$ che rispetti la P.L. $\Rightarrow L$ non è libero

Come doveva farli allarmare? ~~non erano necessari~~

L è l'intersezione di 2 linguaggi context-free:

→ il linguaggio delle stringhe palindromiche

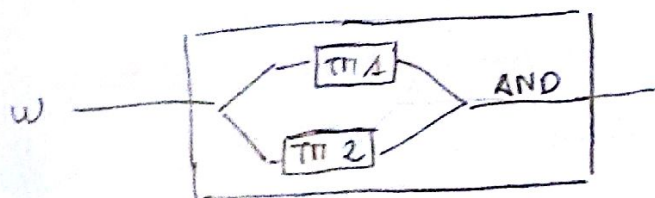
→ il linguaggio con $\#1 \geq \#0$

e l'intersezione di due liberi potrebbe non essere libera.

PASSO 2.1 - Dimostrare che esiste una TM che decide

se il linguaggio una stringa appartiene al linguaggio.

Considero una TM composta da due sotto-TM: una controlla la palindromia, l'altra controlla che gli 1 siano più degli 0



TM1: memorizzo in uno stato il simbolo sulla testina

controlla che il simbolo a destra non sia $\#$ se lo è accetta.

Se non lo è ^{scrivo $\#$ e} li vedo a destra fino al primo $\#$ e poi a sinistra di 1
il confronto il simbolo sulla testina con quello memorizzato negli stati
se sono diversi rifiuto

controlla il correttore a sinistra: se è $\#$ accetta.

Se no torna indietro fino al primo ~~simbolo~~ $\#$ e poi a destra di 1
ed il ciclo ricomincia.

TM2: Controllare che ci siano più 1 che 0: uso TM come PDA:

Uso una TM con 2 nastri. Il secondo nastro è lo stack

inizialmente contiene $\#Z\#$, il push implica una scrittura a destra

Impiego il noto algoritmo di conteggio:

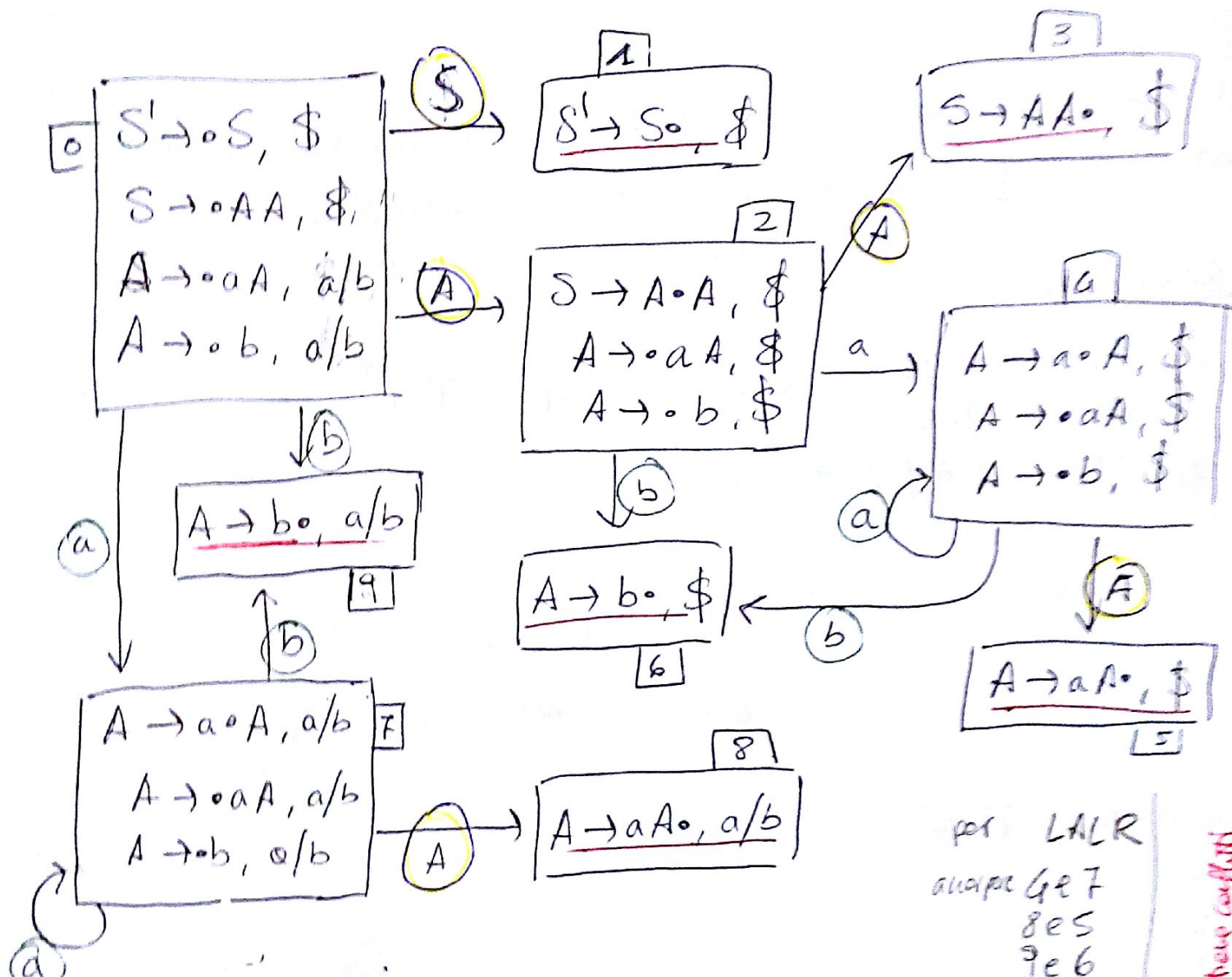
una A per ogni 1 se lo stack è vuoto altrimenti tolgo la B

una B per ogni 0 se lo stack è vuoto altrimenti tolgo la A

accetta se alla fine dell'input ci sono delle A sullo stack
rifiuto se alla fine sullo stack trovo Z o B.

Es 6) LRI

$S \rightarrow AA$ first $\{a, b\}$ follow $\{\$, \}$
 $A \rightarrow aA | b$ first $\{a, b\}$ follow $\{a, b, \$\}$
 $S' \rightarrow S$



per LALR
 merge 4 e 7
 8 e 5
 9 e 6

	a	b	\$	A	S
0	37	99		G2	G1
1			ACC		
2	S4	S6		G3	
3			R: $S \rightarrow AA$		
4	S4	S6		G5	
5			R: $A \rightarrow aA$		
6			R: $A \rightarrow b$		
7	S7	S9		G8	
8	R: $A \rightarrow aA$	R: $A \rightarrow aA$			
9	R: $A \rightarrow b$	R: $A \rightarrow b$			

⇒ la grammatica è LRI
 perché la tabella non contiene conflitti

Es 4) Compificazione linguaggio

Dalla definizione:

$$L = L_1 \cap (L_2 \cdot L_2) \quad | \quad L_1, L_2 \in RE$$

i linguaggi RE sono chiusi rispetto a concatenazione ed ~~concatenazione~~ ^{intersezione}
quindi L è necessariamente RE.

Es 5)

Sì esiste! Ad esempio il CYK de, dato una stringa w
ed un linguaggio L , permette di decidere $w \in L$.
context-free

Descrizione:

Sia G una CNF | $L(G) = L$, $G(V, T, P, S)$
e sia $w = a_1 a_2 \dots a_n$

base $X_{ii} = \{ A \mid A \rightarrow a_i \in P \}$

Induzione $X_{ij} = \{ A \mid A \rightarrow BC \in P, \exists k: i \leq k \leq j, \}$

$B \in X_{ik} \text{ AND } C \in X_{(k+1)j} \}$

$w \in L(G) \Leftrightarrow S \in X_{1n}$